

Deployment & Operations Runbook

dashAI — Saksi Mata Digital yang Netral & Tertandatangani

dashAI

2026-06-21

Daftar Isi

1	Pendahuluan & Cakupan	2
2	Lingkungan & Prasyarat	3
2.1	Topologi lingkungan	3
2.2	Prasyarat tooling	4
2.3	Skrip npm proyek	4
2.4	Konfigurasi build kritis	4
3	Setup Pertama Kali	5
3.1	Langkah GitHub	5
3.2	Membuat proyek Vercel	5
4	Variabel Lingkungan	6
4.1	Perilaku fallback DEV	6
4.2	Generasi DASHAI_SIGNING_KEY	6
4.3	Menyetel env var di Vercel	6
4.4	Rotasi DASHAI_SIGNING_KEY	7
5	Build & Deploy	7
5.1	Pemeriksaan pra-deploy lokal	7
5.2	Deploy ke produksi	7
5.3	Urutan deploy yang disarankan	8
6	Checklist Verifikasi	8
6.1	Route halaman (HTTP 200, bukan 404)	8
6.2	/api/public-key — gerbang isDev: false	8
6.3	End-to-end seal → verify	9
6.4	Laporan PDF (opsional)	9
6.5	Verifikasi di browser	10
7	Prosedur Rollback	10
7.1	Rollback instan (promote deployment sebelumnya)	10
7.2	Rollback via Git	10
7.3	Setelah rollback	10
8	Monitoring & Respons Insiden	11
8.1	Sinyal yang dipantau	11
8.2	Klasifikasi tingkat insiden	12
8.3	Alur respons ringkas	12

8.4	Catatan ketergantungan eksternal	12
9	Masalah Umum	12
9.1	Semua route mengembalikan 404 — <i>framework preset null/other</i>	12
9.2	/api/public-key mengembalikan isDev: true di produksi	13
9.3	Deployment Protection memblokir akses publik / verifikasi gagal	13
9.4	POST /api/report 500 — “Gagal membuat dokumen PDF laporan.”	13
9.5	POST /api/report 422 — envelope tak terverifikasi	13
9.6	POST /api/seal 400 — event tidak lengkap / violation tak dikenal	13
10	Dampak Rotasi Kunci terhadap Laporan yang Sudah Ada	14
10.1	Mekanisme verifikasi	14
10.2	Apa yang terjadi setelah rotasi	14
10.3	Implikasi & mitigasi	15
11	Lampiran	16
11.1	Endpoint server (ringkas)	16
11.2	ViolationKey valid (12) dan pasal UU 22/2009	16
11.3	Perintah cepat	17

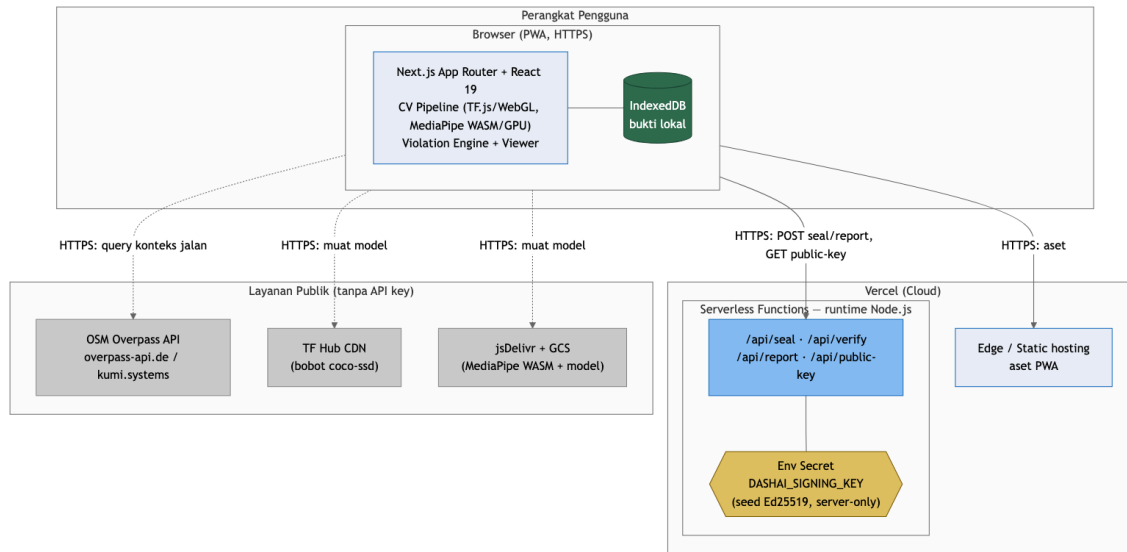
1 Pendahuluan & Cakupan

Dokumen ini adalah *runbook* operasional untuk men-*deploy*, memverifikasi, dan mengoperasikan dashAI di lingkungan produksi. Pembaca yang dituju adalah engineer atau operator yang bertanggung jawab atas rilis ke Vercel dan respons insiden — bukan pengguna akhir.

dashAI adalah aplikasi web **Next.js 16 (App Router)** dengan **React 19, TypeScript (strict)**, dan **Tailwind CSS v4**, yang di-*deploy* di Vercel pada <https://dashai-mu.vercel.app> (sumber: <https://github.com/Finerium/dashAI>, lisensi Apache-2.0). Karakteristik arsitektur yang menentukan seluruh prosedur dalam runbook ini:

- **Inferensi di browser.** Seluruh pipeline computer vision (TensorFlow.js coco-ssd lite_mobilenet_v2, IoU tracker, MediaPipe Tasks face/pose, Tesseract.js OCR pelat) berjalan **di klien**. Bobot model dimuat dari CDN publik saat runtime — tidak ada yang di-*bundle* (lihat `lib/cv/detector.ts`). Akibatnya, server tidak memiliki beban GPU/CV dan tidak menyimpan media.
- **Permukaan server yang sangat kecil.** Hanya empat *route* serverless, semuanya berjalan di **runtime Node.js** (`export const runtime = "nodejs"`): `POST /api/seal`, `POST /api/verify`, `GET /api/public-key`, dan `POST /api/report`. Tidak ada basis data, tidak ada autentikasi pengguna, dan **tidak ada API key pihak ketiga**.
- **Satu-satunya rahasia.** `DASHAI_SIGNING_KEY` — seed `Ed25519` 32-byte (hex) yang dipakai untuk menyegel bukti di `POST /api/seal`. Kunci ini tidak pernah meninggalkan server; kunci publik pasangannya dipublikasikan di `GET /api/public-key`.
- **Penyimpanan lokal-dulu (local-first).** Bukti disimpan di **IndexedDB** di perangkat. Server hanya menerima payload yang **secara eksplisit disegel** oleh pengguna.

Catatan integritas. dashAI bersifat **tamper-evident**, bukan tamper-proof. Tanda tangan membuktikan isi laporan tidak berubah satu byte pun sejak disegel server pada waktu tertera; ia **tidak** membuktikan kamera menyaksikan peristiwa fisik. Demo saat ini berjalan via kamera ponsel; perangkat keras khusus direncanakan pasca-investor.



Gambar 1: Diagram deployment dashAI: browser (PWA/HTTPS) — Vercel (Edge static + Serverless Functions runtime Node.js dengan env secret DASHAI_SIGNING_KEY) — layanan publik tanpa API key (OSM Overpass, TF Hub CDN, MediaPipe CDN).

2 Lingkungan & Prasyarat

2.1 Topologi lingkungan

dashAI memetakan langsung ke tiga jenis lingkungan Vercel. Karena tidak ada basis data dan tidak ada *state* sisi server, perbedaan antar lingkungan hampir seluruhnya berupa nilai DASHAI_SIGNING_KEY.

Lingkungan	Host	DASHAI_SIGNING_KEY	isDev	di /api/ public-key	keyId
Local dev	http:// localhost:3000 (pnpm dev)	Tidak boleh	diset	true (jika tak diset)	dev-...
Preview (Vercel)	URL preview per-branch	Sebaiknya diset (key non- produksi)	false jika diset, true jika tidak		prod-... / dev-...
Production (Vercel)	https:// dashai-mu. vercel.app	Wajib diset	false		prod-...

keyId dibentuk di `lib/crypto/keys.ts` sebagai `("prod-" | "dev-") + pubHex.slice(0, 16)`. Awalan `prod-` menandakan kunci berasal dari env var yang valid; `dev-` menandakan fallback. Verifikator publik dapat membedakan laporan dev dan produksi dari awalan ini.

2.2 Prasyarat tooling

Tool	Versi acuan	Kegunaan
Node.js	20 LTS atau lebih baru (dikembangkan pada Node 25.x lokal)	Build & runtime serverless
pnpm	Terbaru (manajer paket proyek; ada <code>pnpm-lock.yaml</code>)	Instal dependensi, jalankan skrip
Vercel CLI	Terbaru (<code>npm i -g vercel</code>)	vercel deploy, vercel env, vercel logs
Git / GitHub CLI (gh)	Terbaru	Manajemen repositori & integrasi Git Vercel
Browser modern	Chromium/Safari dengan WebGL + WASM + <code>getUserMedia</code>	Demo & verifikasi end-to-end

Akses yang dibutuhkan: akun Vercel dengan akses ke proyek `dashAI`, akses `push/admin` ke repo GitHub `Finerium/dashAI`, dan kemampuan menyetel Environment Variables di Vercel.

2.3 Skrip npm proyek

Didefinisikan di `package.json`:

Skrip	Perintah	Catatan
<code>pnpm dev</code>	<code>next dev</code>	Server pengembangan
<code>pnpm build</code>	<code>next build</code>	Build produksi
<code>pnpm start</code>	<code>next start</code>	Menjalankan hasil build secara lokal
<code>pnpm lint</code>	<code>eslint</code>	Lint (<code>eslint-config-next</code>)
<code>pnpm typecheck</code>	<code>tsc --noEmit</code>	Pemeriksaan tipe (TypeScript strict)

2.4 Konfigurasi build kritis

`next.config.ts` menandai `@react-pdf/renderer` sebagai paket eksternal server agar PDF dapat dirender di serverless Vercel:

```
const nextConfig: NextConfig = {
  serverExternalPackages: ["@react-pdf/renderer"],
};
```

@react-pdf/renderer membawa dependensi mirip-native (fontkit, yoga). Jika konfigurasi ini hilang, POST /api/report bisa gagal saat render PDF di produksi. Jangan menghapus baris ini.

3 Setup Pertama Kali

Bagian ini mengasumsikan repo sudah ada di GitHub dan proyek Vercel belum dibuat (atau perlu dibuat ulang).

3.1 Langkah GitHub

1. Pastikan kode berada di <https://github.com/Finerium/dashAI> dan branch produksi (mis. main) berisi kode yang akan dirilis.
2. Pastikan tidak ada rahasia yang ter-commit. DASHAI_SIGNING_KEY **tidak boleh** ada di repo; .env.example hanya berisi kunci kosong sebagai dokumentasi:

```
DASHAI_SIGNING_KEY=
```

3. Verifikasi .env.local ada di .gitignore (default Next.js).

3.2 Membuat proyek Vercel

Disarankan via Vercel CLI agar dapat diulang dan terdokumentasi:

```
# dari root repo
vercel login
vercel link          # tautkan direktori ke proyek Vercel (buat baru jika
perlu)
```

Saat membuat proyek (CLI atau Dashboard), pastikan pengaturan berikut. **Framework preset WAJIB Next.js** — ini adalah penyebab kegagalan paling umum (lihat Bagian “Masalah Umum”):

Pengaturan	Nilai
Framework Preset	Next.js (jangan biarkan Other/null)
Build Command	<i>Default</i> (next build)
Output Directory	<i>Default</i> (dikelola Next.js)
Install Command	<i>Default</i> (pnpm install, terdeteksi dari pnpm-lock.yaml)
Root Directory	./
Node.js Version	20.x atau lebih baru

Jika menggunakan integrasi Git Vercel (auto-deploy on push), framework preset diset di **Project Settings** → **General** → **Framework Preset**. Jika preset bernilai Other/null, build menghasilkan output yang salah dan situs mengembalikan **404** (semua route, termasuk /).

4 Variabel Lingkungan

dashAI memiliki **tepat satu** variabel lingkungan yang relevan untuk produksi.

Nama	Wajib (prod)	Format	Tujuan
DASHAI_SIGNING_KEY	Ya	Hex 64 karakter (/^[0-9a-fA-F]{64}\$/) — seed Ed25519 32-byte	Kunci privat penandatanganan bukti di /api/seal

Tidak ada variabel lain. Tidak ada kredensial OSM, TensorFlow, MediaPipe, maupun Tesseract — semuanya memakai CDN/endpoint publik tanpa key.

4.1 Perilaku fallback DEV

Logika di lib/crypto/keys.ts:

- Jika DASHAI_SIGNING_KEY diset dan cocok dengan regex hex 64 karakter → kunci dianggap valid, isDev: false, keyId berawalan prod-.
- Jika tidak diset, kosong, atau formatnya salah → dashAI memakai **DEV_KEY_HEX** tetap yang ditandai jelas, isDev: true, keyId berawalan dev-. Laporan yang disegel dengan kunci DEV **tidak boleh** diperlakukan sebagai otoritatif.

Konsekuensi operasional: bila env var di produksi salah ketik atau formatnya tidak valid, aplikasi **tidak crash** — ia diam-diam jatuh ke kunci DEV. Karena itu verifikasi isDev: false pada /api/public-key adalah gerbang rilis wajib (lihat Bagian Verifikasi).

4.2 Generasi DASHAI_SIGNING_KEY

Buat seed Ed25519 32-byte yang aman secara kriptografis (sesuai .env.example dan README):

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Output adalah string hex 64 karakter. Perlakukan sebagai rahasia tingkat tinggi: jangan tampilkan di log, jangan commit, jangan kirim lewat kanal tak terenkripsi.

4.3 Menyetel env var di Vercel

Lewat CLI (disarankan; nilai diminta secara interaktif tanpa muncul di histori shell):

```
# Production
vercel env add DASHAI_SIGNING_KEY production
# (tempel nilai hex saat diminta)

# Optional: Preview & Development
vercel env add DASHAI_SIGNING_KEY preview
vercel env add DASHAI_SIGNING_KEY development
```

Lewat Dashboard: **Project Settings** → **Environment Variables**, tambahkan DASHAI_SIGNING_KEY, tempel nilai hex, pilih *scope* (Production/Preview/Development). Tandai sebagai *Sensitive* bila tersedia.

Penting: Environment Variables hanya berlaku pada *deployment baru*. Setelah menambah/mengubah DASHAI_SIGNING_KEY, Anda **harus me-redeploy** agar nilainya termuat. Kunci di-*cache* di memori serverless (let cached di lib/crypto/keys.ts); deployment baru memulai dari cache kosong.

4.4 Rotasi DASHAI_SIGNING_KEY

Prosedur rotasi (mis. saat dicurigai bocor, atau kebijakan rotasi berkala):

1. Generate seed baru: `node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"`.
2. (Disarankan) Simpan kunci lama di tempat aman terlebih dahulu — diperlukan untuk memverifikasi laporan yang sudah terlanjur terbit (lihat Bagian “Dampak Rotasi Kunci”).
3. Perbarui nilai env var: `vercel env rm DASHAI_SIGNING_KEY production` lalu `vercel env add DASHAI_SIGNING_KEY production`, atau ubah di Dashboard.
4. **Redeploy produksi** (`vercel deploy --prod`).
5. Verifikasi GET `/api/public-key` menampilkan keyId baru (16 hex pertama dari kunci publik baru) dan `isDev: false`.

5 Build & Deploy

5.1 Pemeriksaan pra-deploy lokal

Sebelum men-deploy, jalankan gerbang kualitas yang sama dengan CI:

```
pnpm install --frozen-lockfile
pnpm typecheck      # tsc --noEmit (strict)
pnpm lint           # eslint
pnpm build           # next build — harus sukses tanpa error
```

`pnpm build` harus selesai bersih. Karena CV berjalan di klien dan bobot model dimuat dari CDN saat runtime, build server cepat dan tidak membutuhkan GPU.

5.2 Deploy ke produksi

Alur utama memakai Vercel CLI:

```
# Preview (default) — uji di URL preview lebih dulu
vercel deploy

# Production
vercel deploy --prod
```

Alternatif via integrasi Git: push ke branch produksi (mis. `main`) memicu deployment produksi otomatis; push ke branch lain memicu deployment **Preview**. Apa pun jalurnya, **selalu uji di Preview lebih dahulu** lalu *promote/redeploy* ke produksi.

5.3 Urutan deploy yang disarankan

Langkah	Tindakan	Kriteria lulus
1	pnpm build lokal	Build sukses, 0 error
2	vercel deploy (preview)	URL preview hidup
3	Jalankan checklist verifikasi di URL preview	Semua poin lulus
4	Pastikan DASHAI_SIGNING_KEY produksi terpasang	vercel env ls menampilkannya
5	vercel deploy --prod	Deployment produksi sukses
6	Jalankan checklist verifikasi di domain produksi	Semua poin lulus, terutama isDev:false

6 Checklist Verifikasi

Jalankan setelah setiap deployment ke lingkungan target. Ganti BASE dengan domain yang sedang diuji (mis. <https://dashai-mu.vercel.app>).

6.1 Route halaman (HTTP 200, bukan 404)

Aplikasi mengekspos halaman berikut (app/page.tsx, app/live/page.tsx, app/review/page.tsx, app/verify/page.tsx):

Route	Harapan
GET BASE/	200 (landing)
GET BASE/live	200 (deteksi langsung)
GET BASE/review	200 (peninjauan bukti)
GET BASE/verify	200 (verifikasi mandiri)

```
for p in / /live /review /verify; do
  echo -n "$p -> "; curl -s -o /dev/null -w "%{http_code}\n" "BASE$p"
done
```

Jika **semua** mengembalikan 404, kemungkinan besar *framework preset* salah (lihat Masalah Umum).

6.2 /api/public-key — gerbang isDev: false

Ini adalah pemeriksaan terpenting untuk produksi:

```
curl -s BASE/api/public-key
```

Harapan untuk **produksi**:


```
{
  "publicKeyHex": "<64 hex>",
  "keyId": "prod-<16 hex>",
  "algorithm": "Ed25519",
  "isDev": false
}
```

- isDev **harus** false. Jika true, DASHAI_SIGNING_KEY belum termuat (tidak diset, format salah, atau belum redeploy) — **jangan dirilis sebagai produksi**.
- keyId harus berawalan prod-.
- Respons dilengkapi header Cache-Control: public, max-age=3600.

6.3 End-to-end seal → verify

Bukti kebenaran kriptografis: segel sebuah event lalu verifikasi tanda tangannya. POST /api/seal membutuhkan minimal id, violation (harus ada di katalog CITATIONS), subject (self/other), dan capturedAt (number).

```
# 1) Seal
SEALED=$(curl -s -X POST BASE/api/seal \
  -H "Content-Type: application/json" \
  -d '{"event":{"id":"e2e-check","violation":"lawan-
  arus","subject":"other","capturedAt":1718900000000,"confidence":0.9}}')
echo "$SEALED"

# 2) Verify (kirim kembali envelope di field "sealed")
curl -s -X POST BASE/api/verify \
  -H "Content-Type: application/json" \
  -d '{"sealed": $SEALED}'
```

Harapan:

- Respons seal berisi payload, algorithm: "Ed25519", publicKeyId, payloadHash, signature, sealedAt.
- Respons verify: valid: true, hashMatches: true, signatureValid: true, dengan reason "Tanda tangan sah dan isi laporan utuh (tidak diubah sejak disegel)."

Uji negatif (opsional namun disarankan): ubah satu karakter di payload lalu verifikasi ulang — harus mengembalikan valid: false, hashMatches: false. Ini membuktikan sifat **tamper-evident**.

6.4 Laporan PDF (opsional)

POST /api/report memverifikasi envelope lebih dulu (menolak 422 bila tidak valid), lalu mengembalikan PDF dengan QR menuju /verify. Uji dengan mengirim envelope hasil seal di atas:

```
curl -s -X POST BASE/api/report \
  -H "Content-Type: application/json" \
  -d '{"sealed": $SEALED, "kind": "tilang"}'
```

```
-o report.pdf
file report.pdf # harus "PDF document"
```

Jika `isDev: true`, dokumen tetap dibuat tetapi ditandai sebagai DEV — jangan dipakai sebagai laporan otoritatif.

6.5 Verifikasi di browser

Buka `BASE/verify` di browser. Halaman ini mengambil kunci publik dari `/api/public-key` dan memverifikasi envelope **sepenuhnya di sisi klien**, sehingga kepercayaan tidak bergantung pada UI dashAI. Konfirmasi tidak ada error konsol dan status verifikasi tampil benar (VALID / DIUBAH).

7 Prosedur Rollback

Karena dashAI **tanpa basis data dan tanpa migrasi**, rollback bersih dan cepat: cukup mengembalikan deployment, tanpa langkah pemulihan data.

7.1 Rollback instan (promote deployment sebelumnya)

Cara tercepat — kembalikan ke deployment produksi yang baik sebelumnya:

```
# Lihat deployment terakhir, catat URL yang stabil sebelumnya
vercel ls
```

```
# Promote deployment lama menjadi produksi
vercel promote <DEPLOYMENT_URL>
```

Alternatif melalui Dashboard: **Deployments** → **pilih deployment yang sehat** → **Promote to Production** (atau “Instant Rollback” bila tersedia). Operasi ini mengarahkan domain produksi ke build lama yang sudah jadi — tidak ada rebuild, sehingga berlaku dalam hitungan detik.

7.2 Rollback via Git

Bila masalah ada di kode dan ingin memperbaiki garis sejarah:

```
git revert <commit_buruk> # atau perbaiki langsung
git push origin main      # memicu deployment produksi baru (jika auto-deploy aktif)
# atau
vercel deploy --prod
```

7.3 Setelah rollback

1. Jalankan **Checklist Verifikasi** pada produksi.
2. Pastikan `/api/public-key` tetap `isDev: false` dan `keyId` sesuai harapan — pastikan rollback tidak mengembalikan ke deployment yang dibuat sebelum env var diset.
3. Catat insiden (lihat Bagian Monitoring & Respons Insiden).

Peringatan rotasi: Jangan me-rollback ke deployment yang dibuat *sebelum* rotasi kunci jika kunci lama sudah dicabut karena alasan keamanan — deployment lama akan menyegel

dengan kunci yang sudah dianggap tidak tepercaya. Untuk kasus kebocoran kunci, jalur yang benar adalah **roll-forward** (deploy ulang dengan kunci baru), bukan rollback.

8 Monitoring & Respons Insiden

8.1 Sinyal yang dipantau

Tanpa beban server yang berat, pemantauan terfokus pada empat *route* dan kesehatan kunci.

Sinyal	Sumber	Kondisi sehat
Status route halaman	Vercel Analytics / uji curl	200 untuk /, /live, /review, /verify
isDev produksi	GET /api/public-key	false
keyId produksi	GET /api/public-key	berawalan prod-, sesuai kunci aktif
Error fungsi serverless	Vercel → Logs Observability	/ Tidak ada lonjakan 5xx pada /api/seal, /api/report, /api/verify
Tingkat 4xx /api/seal	Vercel Logs	Rendah; 400 berarti payload klien tidak lengkap/violation tak dikenal
Render PDF	/api/report	Tidak ada 500 (“Gagal membuat dokumen PDF laporan.”)

Akses log: `vercel logs <deployment-url>` atau panel **Observability** di Dashboard.

8.2 Klasifikasi tingkat insiden

Severity	Definisi	Contoh
SEV-1	Produksi menerbitkan bukti yang tidak tepercaya, atau seluruh situs down	/ api/public-key isDev:true di produksi; semua route 404; kecurigaan kebocoran DASHAI_SIGNING_KEY
SEV-2	Fungsi inti rusak sebagian	/api/seal atau /api/report mengembalikan 5xx; verifikasi gagal pada laporan sah
SEV-3	Degradasi non-kritis	Latensi tinggi; gangguan CDN model (deteksi gagal dimuat di klien); Overpass lambat

8.3 Alur respons ringkas

1. **Triage** — identifikasi route/sinyal yang terdampak dan severity.
2. **Stabilkan** — untuk regresi rilis, lakukan **rollback instan** (promote deployment sebelumnya). Untuk isDev:true di produksi, perbaiki env var lalu **redeploy** (roll-forward).
3. **Kecurigaan kebocoran kunci (SEV-1)** — segera **rotasi DASHAI_SIGNING_KEY** (lihat prosedur rotasi) dan redeploy. Pahami dampaknya pada laporan lama (bab berikut).
4. **Verifikasi** — jalankan Checklist Verifikasi pada produksi.
5. **Dokumentasikan** — catat lini masa, penyebab, dan tindakan; perbarui runbook bila ditemukan celah prosedur.

8.4 Catatan ketergantungan eksternal

Inferensi klien bergantung pada CDN publik (TF Hub untuk bobot coco-ssd, jsDelivr/GCS untuk MediaPipe WASM+model) dan **OSM Overpass** untuk konteks jalan (oneway, maxspeed). Insiden CDN/Overpass memengaruhi pengalaman deteksi di klien, **bukan** integritas penyegelan: /api/seal dan /api/verify tetap berfungsi karena tidak bergantung pada layanan eksternal tersebut. Klasifikasikan gangguan demikian sebagai SEV-3 kecuali ada dampak lebih luas.

9 Masalah Umum

9.1 Semua route mengembalikan 404 — *framework preset null/0ther*

Gejala: /, /live, /review, /verify semua 404 setelah deploy.

Penyebab: Framework preset proyek Vercel tidak diset ke **Next.js** (bernilai 0ther/null). Vercel tidak menjalankan build Next.js dengan benar dan tidak memetakan App Router, sehingga tidak ada route yang ter-resolve.

Perbaikan: **Project Settings** → **General** → **Framework Preset** → **Next.js**, simpan, lalu **redeploy**. Verifikasi ulang dengan loop curl route halaman.

9.2 **/api/public-key** mengembalikan **isDev: true** di produksi

Gejala: Endpoint berfungsi tetapi **isDev: true** dan **keyId** berawalan **dev-**.

Penyebab: **DASHAI_SIGNING_KEY** tidak diset untuk scope **Production**, salah ketik, formatnya bukan hex 64 karakter, atau **env var** diubah tetapi belum di-**redeploy**.

Perbaikan: Pastikan nilai cocok `^[0-9a-fA-F]{64}$`, set untuk scope **Production** (`vercel env add DASHAI_SIGNING_KEY production`), lalu `vercel deploy --prod`. Konfirmasi **isDev: false**.

9.3 **Deployment Protection** memblokir akses publik / verifikasi gagal

Gejala: Pengunjung atau verifikator publik mendapat halaman autentikasi Vercel; QR **/verify** tidak dapat dibuka oleh pihak ketiga; uji curl mendapat 401.

Penyebab: **Vercel Deployment Protection** (mis. **Vercel Authentication / Password Protection / SSO**) aktif pada deployment. Karena verifikasi dashAI harus dapat dilakukan **siapa pun** (kunci publik di **/api/public-key**, halaman **/verify** mandiri), proteksi yang memblokir akses anonim merusak fungsi inti.

Perbaikan: Untuk domain produksi yang ditujukan ke publik, nonaktifkan **Deployment Protection** pada lingkungan **Production** (**Project Settings** → **Deployment Protection**), atau gunakan *Protection Bypass for Automation* untuk pengujian otomatis. Pastikan minimal **/api/public-key** dan **/verify** dapat diakses anonim. Preview boleh tetap dilindungi.

9.4 **POST /api/report 500** — “Gagal membuat dokumen PDF laporan.”

Gejala: Seal & verify normal, tetapi pembuatan PDF gagal 500 di produksi.

Penyebab: `@react-pdf/renderer` ter-bundle oleh kompiler server. Konfigurasi `serverExternalPackages:["@react-pdf/renderer"]` di `next.config.ts` hilang/berubah.

Perbaikan: Kembalikan baris tersebut di `next.config.ts`, rebuild, redeploy.

9.5 **POST /api/report 422** — **envelope** tak terverifikasi

Gejala: Permintaan report ditolak 422 dengan reason berisi pesan verifikasi.

Penyebab: Ini **bukan bug** — **/api/report** sengaja menolak **envelope** yang tidak valid (hash tidak cocok / tanda tangan tidak sah) agar laporan otoritatif tidak dibuat dari bukti yang rusak. Sering terjadi bila **envelope** diubah, atau bila disegel oleh kunci yang berbeda dari kunci server saat ini (mis. setelah rotasi).

Perbaikan: Segel ulang event dengan instance produksi saat ini, atau verifikasi memakai kunci publik yang sesuai.

9.6 **POST /api/seal 400** — **event** tidak lengkap / **violation** tak dikenal

Gejala: Seal menolak 400.

Penyebab: Payload kurang id/violation/subject/capturedAt, subject bukan self/other, capturedAt bukan number, atau violation tidak ada di CITATIONS. Katalog violation yang sah persis 12 kunci (lihat Lampiran).

Perbaikan: Kirim payload yang sesuai kontrak. Gunakan salah satu dari 12 ViolationKey valid.

10 Dampak Rotasi Kunci terhadap Laporan yang Sudah Ada

Memahami konsekuensi rotasi DASHAI_SIGNING_KEY penting karena verifikasi bersifat **stateless** dan **self-contained**: envelope tersegel (dan PDF) membawa seluruh data verifikasi di dalamnya (QR/URL /verify?d=<base64url envelope>), tanpa lookup basis data.

10.1 Mekanisme verifikasi

verifySealed (di lib/crypto/verify.ts) memeriksa dua hal: (1) hash kanonik payload cocok; (2) tanda tangan Ed25519 sah **untuk kunci publik yang diberikan**. Halaman /verify mengambil kunci publik dari /api/public-key instance dashAI, yaitu kunci **yang sedang aktif**.

10.2 Apa yang terjadi setelah rotasi

- Laporan **baru** ditandatangani dengan kunci baru → verifikasi dengan kunci publik baru: **VALID**.
- Laporan **lama** ditandatangani dengan kunci lama. Saat diverifikasi terhadap kunci **baru** (kunci default yang dipublikasikan /api/public-key setelah rotasi), tanda tangan tidak akan cocok → signatureValid: false → hasil **DIUBAH/INVALID**, meskipun isinya tidak pernah diubah.
- Penting: hal ini **tidak** berarti laporan lama dipalsukan. Hanya berarti kunci verifikasi yang dipakai bukan kunci penandatangannya.

10.3 Implikasi & mitigasi

Konteks	Implikasi	Mitigasi
Kebocoran kunci (kunci lama harus dicabut)	Laporan lama menjadi “tidak dapat diverifikasi” terhadap kunci aktif	Inilah perilaku yang diinginkan: laporan yang disegel kunci bocor tidak lagi tepercaya. Terbitkan ulang bukti penting dengan kunci baru bila masih dibutuhkan dan tersedia sumbernya.
Rotasi rutin/preventif (kunci lama tidak bocor)	Laporan lama tetap sah secara historis tetapi gagal verifikasi terhadap kunci aktif	Arsipkan kunci publik lama dan <code>keyId</code> -nya. Verifikator dapat memeriksa laporan lama terhadap kunci publik lama yang diarsipkan (verifikasi <code>verifySealed</code> menerima <code>publicKeyHex</code> apa pun). <code>keyId</code> pada envelope (<code>prod-<16 hex></code>) menunjukkan kunci mana yang dipakai.
Identifikasi kunci	Setiap envelope membawa <code>publicKeyId</code>	Petakan <code>publicKeyId</code> ke kunci publik yang sesuai untuk memilih kunci verifikasi yang benar.

Rekomendasi: Perlakukan rotasi sebagai peristiwa yang jarang dan terencana. Selalu **arsipkan kunci publik (hex) lama beserta `keyId`** sebelum mengganti env var, agar laporan historis yang sah tetap dapat diverifikasi terhadap kunci aslinya. Untuk skenario kebocoran, rotasi justru bertujuan menjadikan laporan kunci-bocor tidak lagi valid terhadap kunci tepercaya saat ini.

11 Lampiran

11.1 Endpoint server (ringkas)

Endpoint	Method	Runtime	Fungsi
/api/seal	POST	nodejs	Menyegel payload bukti (Ed25519); stempel waktu server + sitasi hukum + tanda tangan
/api/verify	POST	nodejs	Verifikasi envelope sisi server (kenyamanan; / verify juga verifikasi di klien)
/api/public-key	GET	nodejs	Mempublikasikan kunci publik Ed25519, keyId, algorithm, isDev
/api/report	POST	nodejs	Render PDF tertandatangan (tilang/kecelakaan/coaching) + QR ke / verify; menolak 422 bila envelope tak valid

11.2 ViolationKey valid (12) dan pasal UU 22/2009

Konsisten dengan lib/legal/citations.ts (12/12 sitasi terverifikasi, verifikasi adversarial 3-voter). Hanya nilai-nilai inilah yang diterima oleh /api/seal dan /api/report.

ViolationKey	Tier	Pasal (UU 22/2009)
lawan-arus	core	Pasal 287 ayat (1)
tanpa-helm	core	Pasal 291 ayat (1)
terobos-lampu-merah	core	Pasal 287 ayat (2)
boncengan-lebih	core	Pasal 292 ayat (9)
melebihi-kecepatan	core	Pasal 287 ayat (5)
langgar-marka	secondary	Pasal 287 ayat (1)
tanpa-plat	secondary	Pasal 280 ayat (1)
tanpa-lampu-malam	secondary	Pasal 293 ayat (1)
motor-lampu-siang	secondary	Pasal 293 ayat (2)
penumpang-tanpa-helm	secondary	Pasal 291 ayat (2)
tanpa-sabuk	cabin	Pasal 289 ayat (6)
main-hp	cabin	Pasal 283 ayat (1)

Deteksi live yang aktif saat ini: **lawan arah** (OSM oneway + arah arus visual), **ngebut diri sendiri** (GPS vs OSM maxspeed, akurat), **boncengan**, dan **terobos lampu merah** (lihat lib/violations/engine.ts). Pelanggaran lain dikatalogkan dan ditampilkan via dataset demo; deteksi penuh (helm/pelat/kabin) membutuhkan model khusus / kamera kabin.

11.3 Perintah cepat

```
# Generate seed Ed25519 (DASHAI_SIGNING_KEY)
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"

# Set env var produksi + deploy
vercel env add DASHAI_SIGNING_KEY production
vercel deploy --prod

# Cek gerbang produksi
curl -s https://dashai-mu.vercel.app/api/public-key # harus isDev:false, keyId prod-...

# Rollback instan
vercel ls
vercel promote <DEPLOYMENT_URL>
```

Disclaimer. dashAI adalah demonstrasi teknologi. Laporan yang dihasilkan **bukan** dokumen resmi kepolisian; sitasi bersifat indikatif; estimasi kecepatan kendaraan lain bersifat perkiraan. dashAI bersifat **tamper-evident**, bukan tamper-proof.